

Министерство образования Российской Федерации

**МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ
им. Н.Э. БАУМАНА**

Факультет: Информатика и системы управления
Кафедра: Информационная безопасность (ИУ8)

**ИНТЕЛЛЕКТУАЛЬНЫЕ ТЕХНОЛОГИИ
ИНФОРМАЦИОННОЙ БЕЗОПАСНОСТИ**

Лабораторная работа №1 на тему:
**«Исследование однослойных нейронных сетей на примере
моделирования булевых выражений»**

Вариант 13

Преподаватель:
Коннова Н.С.

Студент:
Степанова Е.А.

Группа:
ИУ8-21М

Москва 2026

Цель работы

Исследовать функционирование простейшей нейронной сети (НС) на базе нейрона с нелинейной функцией активации и ее обучение по правилу Видроу-Хоффа.

Постановка задачи

Получить модель булевой функции (БФ) на основе однослойной НС (единичный нейрон) с двоичными входами $x_1, x_2, x_3, x_4 \in \{0,1\}$, единичным входом смещения $x_0 = 1$, синаптическими весами w_0, w_1, w_2, w_3, w_4 , двоичным выходом $y \in \{0,1\}$ и заданной нелинейной функцией активации (ФА) $f: R \rightarrow (0,1)$.

Для заданной БФ реализовать обучение НС для двух случаев:

1. с использованием всех комбинаций переменных x_1, x_2, x_3, x_4 ;
2. с использованием части возможных комбинаций переменных x_1, x_2, x_3, x_4 ; остальные комбинации используются в качестве тестовых.

Ход работы

Получим таблицу истинности для моделируемой БФ:

$$(\bar{x}_1 + \bar{x}_2 + \bar{x}_3)(\bar{x}_2 + \bar{x}_3 + x_4)$$

x_1	x_2	x_3	x_4	$\bar{x}_1$	$\bar{x}_2$	$\bar{x}_3$	$\bar{x}_1 + \bar{x}_2 + \bar{x}_3$	$\bar{x}_2 + \bar{x}_3 + x_4$	F
0	0	0	0	1	1	1	1	1	1
0	0	0	1	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1
0	0	1	1	1	1	0	1	1	1
0	1	0	0	1	0	1	1	1	1
0	1	0	1	1	0	1	1	1	1
0	1	1	0	1	0	0	1	0	0
0	1	1	1	1	0	0	1	1	1
1	0	0	0	0	1	1	1	1	1
1	0	0	1	0	1	1	1	1	1
1	0	1	0	0	1	0	1	1	1
1	0	1	1	0	1	0	1	1	1
1	1	0	0	0	0	1	1	1	1
1	1	0	1	0	0	1	1	1	1
1	1	1	0	0	0	0	0	0	0
1	1	1	1	0	0	0	0	1	0

На начальном шаге $l = 0$ (эпоха $k = 0$) весовые коэффициенты берутся в виде:

$$w_0^{(0)} = w_1^{(0)} = w_2^{(0)} = w_3^{(0)} = w_4^{(0)} = 0$$

Норма обучения для всех случаев выбирается $\eta = 0.3$

1. Обучение НС с использованием всех комбинаций переменных x_1, x_2, x_3, x_4 .

1.1. Используя пороговую ФА:

$$f(net) = \begin{cases} 1, & net > 0, \\ 0, & net \leq 0 \end{cases}$$

Таблица 1 Параметры НС на последовательных эпохах (пороговая ФА)

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	Y = (0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), W = (0.00, 0.00, 0.00, 0.00, 0.00), E = 3
1	Y = (1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0), W = (-0.30, -0.30, -0.30, -0.30, 0.30), E = 7
2	Y = (1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0), W = (0.00, -0.60, -0.60, -0.60, 0.30), E = 6
3	Y = (1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0), W = (0.60, -0.60, -0.60, -0.30, 0.60), E = 6
4	Y = (1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0), W = (0.60, -0.90, -0.90, -0.60, 0.60), E = 5
5	Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0), W = (0.90, -0.90, -1.20, -0.60, 0.30), E = 3
6	Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0), W = (1.20, -1.20, -0.90, -0.60, 0.30), E = 3
7	Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0), W = (1.50, -1.20, -0.60, -0.60, 0.60), E = 4
8	Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0), W = (1.50, -0.90, -0.90, -0.90, 0.60), E = 2

9	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0),$ $W = (1.50, -0.90, -1.20, -0.90, 0.30), E = 2$
10	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 0),$ $W = (1.50, -1.20, -1.20, -0.90, 0.30), E = 3$
11	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0),$ $W = (1.80, -1.20, -0.90, -0.90, 0.60), E = 4$
12	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0),$ $W = (1.80, -1.20, -1.20, -0.90, 0.60), E = 2$
13	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0),$ $W = (1.80, -1.20, -1.50, -0.90, 0.30), E = 3$
14	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0),$ $W = (2.10, -1.20, -1.20, -0.90, 0.60), E = 4$
15	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0),$ $W = (2.10, -1.20, -1.20, -1.20, 0.60), E = 2$
16	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.10, -1.20, -1.50, -1.20, 0.30), E = 1$
17	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.40, -1.20, -1.20, -0.90, 0.60), E = 2$
18	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0),$ $W = (2.40, -0.90, -1.50, -0.90, 0.60), E = 3$
19	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0),$ $W = (2.10, -1.20, -1.80, -1.20, 0.60), E = 3$
20	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.40, -1.20, -1.50, -1.20, 0.60), E = 1$
21	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0),$ $W = (2.70, -0.90, -1.20, -1.20, 0.60), E = 3$

22	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.40, -1.20, -1.50, -1.50, 0.60), E = 1$
23	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.70, -1.20, -1.20, -1.20, 0.90), E = 2$
24	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (2.70, -0.90, -1.50, -1.20, 0.90), E = 0$

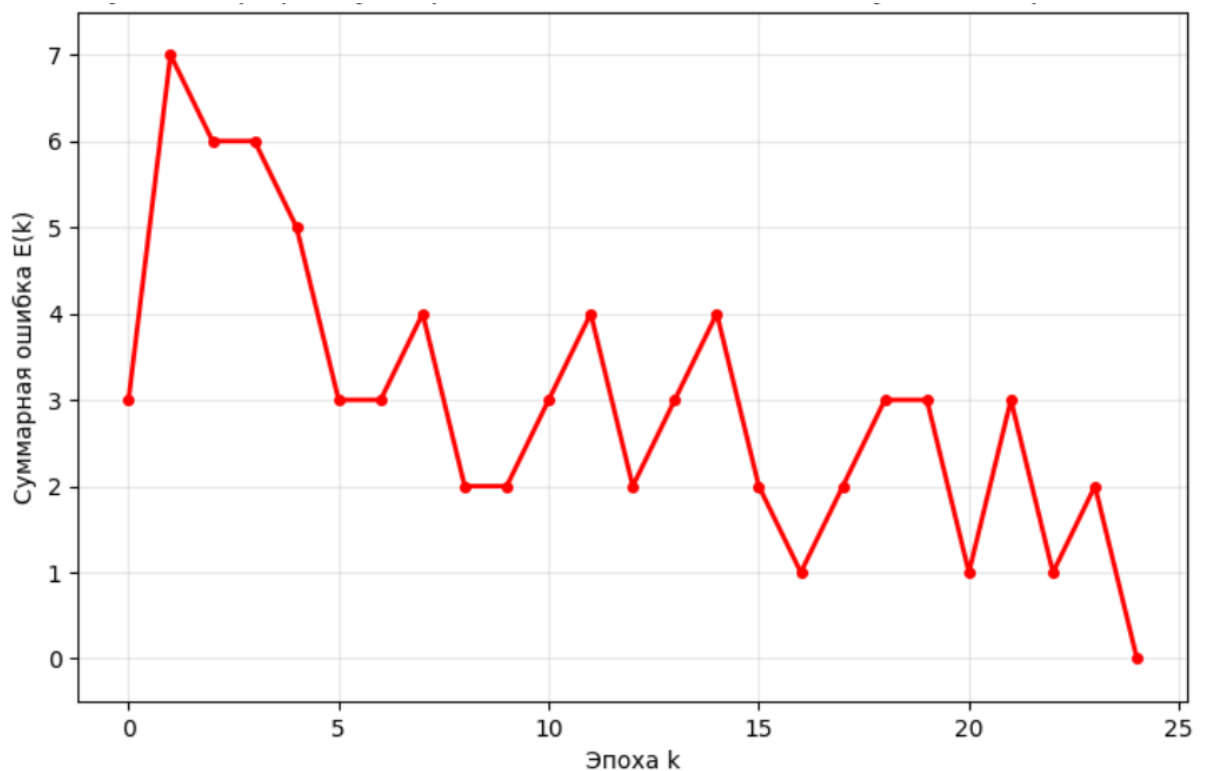


Рисунок 1 График суммарной ошибки НС по эпохам обучения (пороговая ФА)

1.2. Используя сигмоидальную (логистическую) ФА:

$$f(\text{net}) = \frac{1}{2} \left(\frac{\text{net}}{1 + |\text{net}|} + 1 \right)$$

производная, которой

$$\frac{df(\text{net})}{d\text{net}} = \frac{0.5}{(1 + [\text{net}])^2}$$

Таблица 2 Параметры НС на последовательных эпохах (логистическая ФА)

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	$Y = (1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (0.0, 0.0, 0.0, 0.0, 0.0), E = 3$
1	$Y = (1, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0),$ $W = (0.01, 0.18, -0.17, -0.17, 0.08), E = 6$
2	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1),$ $W = (0.09, 0.19, -0.28, -0.33, 0.06), E = 6$
3	$Y = (1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0),$ $W = (0.38, 0.01, -0.15, -0.28, 0.18), E = 1$
4	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0),$ $W = (0.19, -0.18, -0.34, -0.47, -0.02), E = 5$
5	$Y = (1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0),$ $W = (0.46, -0.31, -0.23, -0.45, 0.28), E = 2$
6	$Y = (1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0),$ $W = (0.44, -0.33, -0.40, -0.46, 0.11), E = 5$
7	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0),$ $W = (0.54, -0.35, -0.74, -0.37, 0.23), E = 3$
8	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0),$ $W = (0.78, -0.29, -0.76, -0.30, 0.23), E = 2$
9	$Y = (1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0),$ $W = (0.79, -0.50, -0.74, -0.29, 0.45), E = 2$
10	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 0),$ $W = (0.69, -0.60, -0.84, -0.51, 0.23), E = 3$
11	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0),$ $W = (0.94, -0.77, -0.59, -0.45, 0.29), E = 3$
12	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0),$ $W = (1.12, -0.58, -0.57, -0.47, 0.29), E = 4$

13	$Y = (1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0),$ $W = (1.06, -0.59, -0.86, -0.52, 0.45), E = 2$
14	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0),$ $W = (1.13, -0.52, -1.02, -0.46, 0.28), E = 2$
15	$Y = (1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0),$ $W = (1.19, -0.68, -0.96, -0.40, 0.35), E = 1$
16	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (1.31, -0.56, -0.84, -0.40, 0.35), E = 4$
17	$Y = (1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0),$ $W = (1.33, -0.50, -0.82, -0.56, 0.40), E = 0$

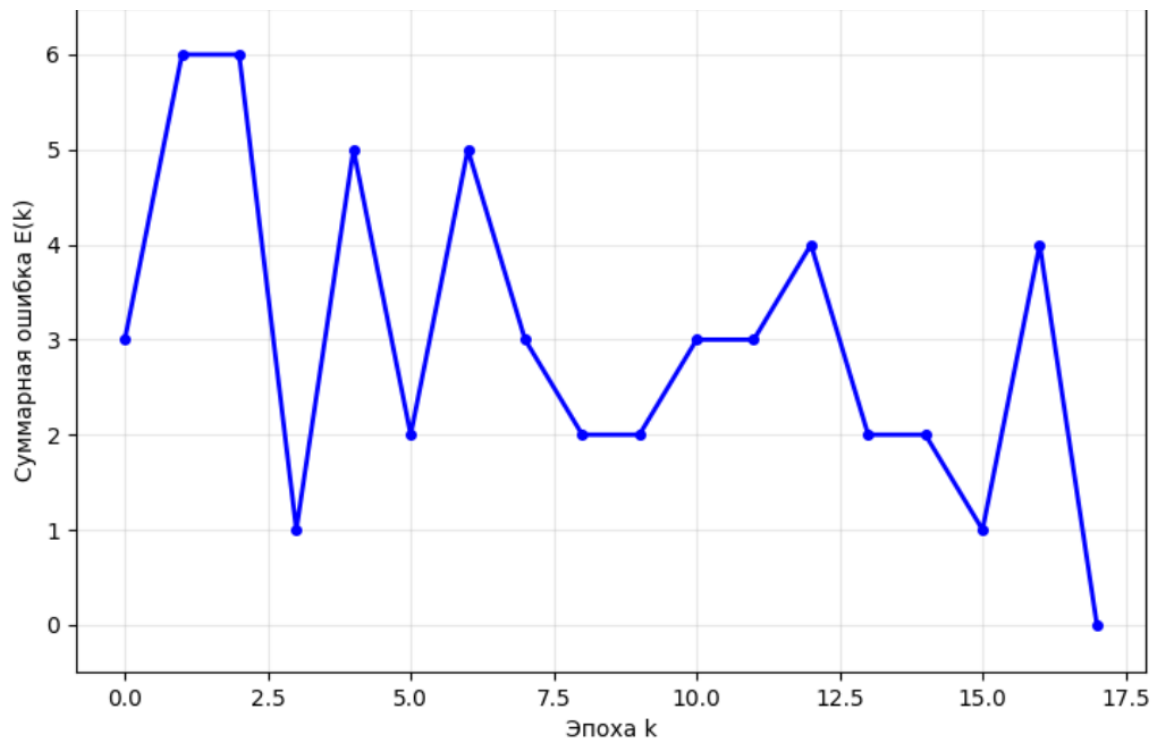


Рисунок 2 График суммарной ошибки НС по эпохам обучения (логистическая ФА)

2. Обучение НС с использованием части комбинаций переменных x_1 , x_2, x_3, x_4 .

Последовательно уменьшая выборку количества векторов, найдём наименьшее количество необходимых для обучения векторов.

2.1. Используя пороговую ФА.

Минимальный набор из четырёх векторов:

$$X^{(1)} = (0,0,0,1), X^{(2)} = (0,1,0,0), X^{(3)} = (0,1,1,0), X^{(4)} = (1,1,1,1)$$

Даёт следующие синаптические коэффициенты:

$$W = (0.00, -0.30, -0.30, -0.30, 0.00)$$

Для полного обучения потребовалось 1 эпоху.

Таблица 3 Параметры НС на последовательных эпохах (пороговая ФА) при наборе из 4 векторов

Номер эпохи, k	Вектор весов W выходной вектор Y, суммарная ошибка E
0	$Y = (0, 0, 1, 1),$ $W = (0.0, 0.0, 0.0, 0.0, 0.0), E = 2$
1	$Y = (0, 0, 1, 1),$ $W = (0.00, -0.30, -0.30, -0.30, 0.00), E = 0$

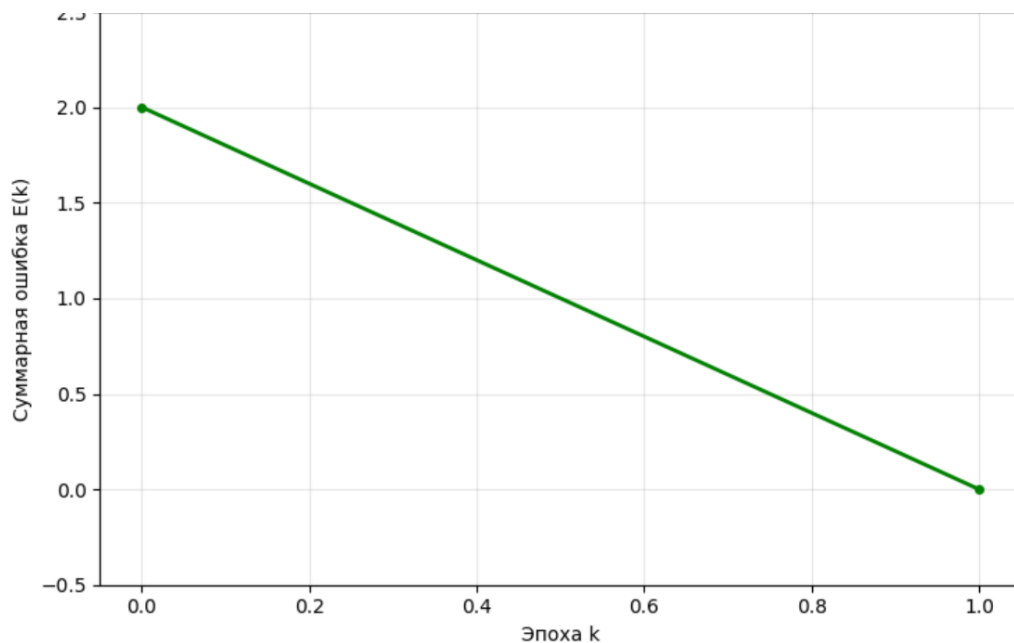


Рисунок 3 График суммарной ошибки НС по эпохам обучения (пороговая ФА) при наборе из 4-х векторов

2.2. Используя сигмоидальную (логистическую) ФА.

Минимальный набор из четырёх векторов:

$$X^{(1)} = (0,0,1,1), X^{(2)} = (1,0,0,0), X^{(3)} = (1,1,0,0), X^{(4)} = (1,1,1,1)$$

Даёт следующие синаптические коэффициенты:

$$W = (0.06, -0.57, -0.10, -0.10, 0.15)$$

Для полного обучения потребовалось 4 эпохи.

Таблица 4 Параметры НС на последовательных эпохах (логистическая ФА) при наборе из 4 векторов

Номер эпохи, k	Вектор весов W, выходной вектор Y, суммарная ошибка E
0	$Y = (0, 0, 1, 0), W = (0.0, 0.0, 0.0, 0.0, 0.0), E = 3$
1	$Y = (0, 1, 1, 1), W = (0.01, -0.25, -0.15, -0.15, 0.10), E = 1$
2	$Y = (0, 1, 1, 1), W = (0.18, -0.25, 0.02, 0.02, 0.27), E = 2$
3	$Y = (0, 0, 1, 1), W = (0.22, -0.41, 0.06, 0.06, 0.31), E = 1$
4	$Y = (0, 0, 1, 1), W = (0.06, -0.57, -0.10, -0.10, 0.15), E = 0$

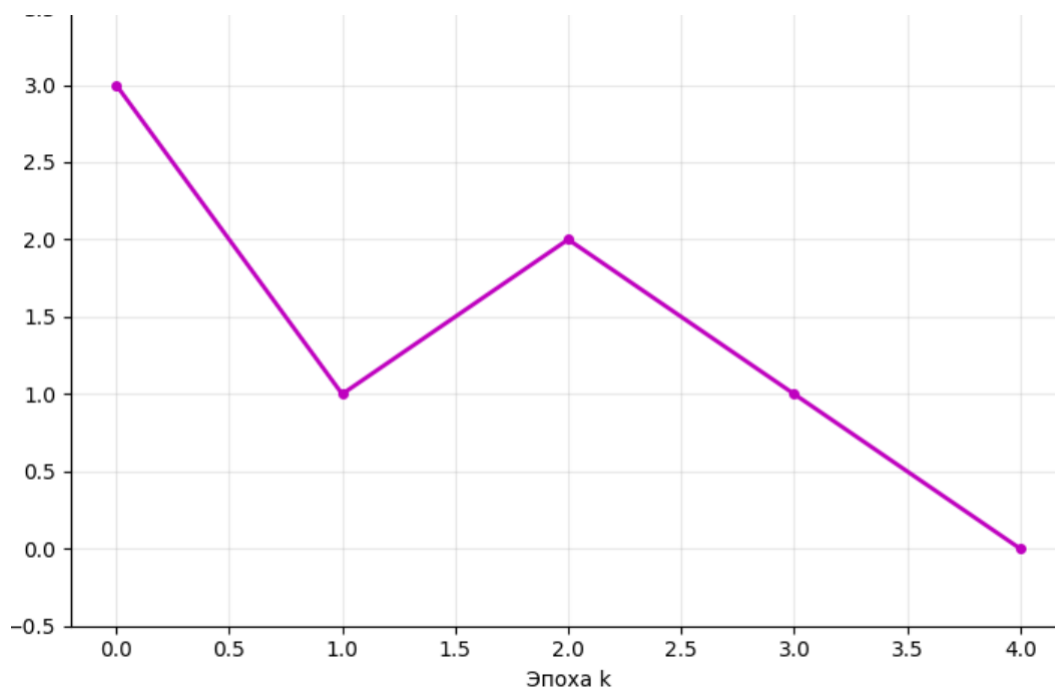


Рисунок 4 График суммарной ошибки НС по эпохам обучения (логистическая ФА) при наборе из 4-х векторов

Выводы

В ходе проделанной работы была изучена работа простейшего нейрона НС на базе нейрона с нелинейной функцией активации и проведено обучение по правилу Видроу-Хоффа. В качестве функций активации брались две различные функции – пороговая и логистическая. В ходе обучения на полных наборах было выявлено, что с использованием логистической функции активации понадобилось меньше эпох, чем для обучения с использованием пороговой функции активации.

Кроме того, для случаев пороговой и логистической функций активации были найдены минимально возможные наборы векторов, на которых можно обучить НС. В обоих случаях удалось найти наборы, состоящие из четырёх векторов. В случае обучения с использованием пороговой функции активации понадобилось меньшее количество эпох, чем с использованием логистической.

Приложение А.

Файл 'lab1.py'.

```
...
import numpy as np
import matplotlib.pyplot as plt
from typing import List, Tuple, Optional

class Neuron:

    def __init__(self, activation_type: str = 'threshold'):

        # w = (w0, w1, w2, w3, w4)
        self.weights = np.zeros(5) # [0,0,0,0,0]
        self.act_type = activation_type
        self.loss_history = [] # История ошибок по эпохам
        self.weights_history = [] # История весов
        self.output_history = [] # История выходных векторов

    def activate(self, net: float) -> float:

        if self.act_type == 'threshold':
            # f(net) = { 1, net >= 0; 0, net < 0 } пороговая
            return 1.0 if net >= 0 else 0.0
        else:
            # f(net) = 1/2 * (net/(1+|net|) + 1) логическая
            return 0.5 * (net / (1 + abs(net)) + 1)

    def derivative(self, net: float) -> float:

        if self.act_type == 'threshold':
```

```

        # В случае НС с пороговой ФА коррекцию веса следует брать в
        # виде  $\Delta w = \eta \cdot \delta \cdot x$ 
        return 1.0
    else:
        # Производная:  $df(net)/d net = 0.5/(1+|net|)^2$ 
        return 0.5 / ((1 + abs(net)) ** 2)

def forward(self, x: List[int], continuous: bool = False) -> float:

    # Добавляем  $x_0 = 1$  для смещения
    x_with_bias = [1.0] + [float(v) for v in x]

    #  $net = \sum w_i \cdot x_i + w_0$ 
    net = sum(w * xi for w, xi in zip(self.weights, x_with_bias))

    # Применяем функцию активации
    out = self.activate(net)

    if continuous:
        return out

    #  $y(out) = \{ 1, out \geq 0.5; 0, out < 0.5 \}$ 
    return 1.0 if out >= 0.5 else 0.0

def predict_all(self, X: List[List[int]]) -> List[int]:    #
    Предсказать выходы для всех входов
    return [int(self.forward(x)) for x in X]

def train_step(self, x: List[int], target: int, eta: float) -> float:
    # Один шаг обучения по правилу Видроу-Хоффа

    # Прямой проход
    x_with_bias = [1.0] + [float(v) for v in x]
    net = sum(w * xi for w, xi in zip(self.weights, x_with_bias))
    out_cont = self.activate(net)
    output = 1.0 if out_cont >= 0.5 else 0.0

    #  $\delta = t - y$ 
    delta = target - output

    # Правило Видроу-Хоффа (дельта-правило)
    #  $w_i^{(l+1)} = w_i^{(l)} + \Delta w_i^{(l)}$ 
    #  $\Delta w_i^{(l)} = \eta \cdot \delta^{(l)} \cdot (df(net)/d net) \cdot x_i^{(l)}$ 
    if abs(delta) > 1e-6:
        df = self.derivative(net)    # Производная функции активации
        for i in range(5):
            self.weights[i] += eta * delta * df * x_with_bias[i]

    return abs(delta)

```

```

def train_epoch(self, X: List[List[int]], targets: List[int], eta:
float) -> Tuple[float, List[int]]:

    epoch_error = 0.0

    for x, t in zip(X, targets):
        epoch_error += self.train_step(x, t, eta)

    # После обновления весов вычисляем выходы для всех образцов
    outputs = self.predict_all(X)
    return epoch_error, outputs

def train(self, X: List[List[int]], targets: List[int],
eta: float, max_epochs: int = 50, verbose: bool = False,
experiment_type: str = "full") -> List[float]:

    # На каждой эпохе вычисляется суммарная квадратичная ошибка  $E(k)$ 

    self.loss_history = []
    self.weights_history = [self.weights.copy()]
    self.output_history = []

    for epoch in range(max_epochs):
        error, outputs = self.train_epoch(X, targets, eta)
        self.loss_history.append(error)
        self.weights_history.append(self.weights.copy())
        self.output_history.append(outputs)

        if verbose:
            # Форматируем выходной вектор как в методичке (первые 8
значений)

            if experiment_type == "full" and len(outputs) > 8:
                # Для полной выборки показываем первые 8 значений
                first_eight = outputs[:16]
                outputs_str = f"({'', ' '.join(map(str, first_eight))},
...) "

            else:
                # Для минимального набора показываем все значения
                outputs_str = f"({'', ' '.join(map(str, outputs))}) "

            w = self.weights
            print(f"Эпоха {epoch:2d}: ошибка = {error:.2f}, "
                  f"веса = ({w[0]:.2f}, {w[1]:.2f}, {w[2]:.2f},
{w[3]:.2f}, {w[4]:.2f}), "
                  f"выход = {outputs_str}")

        if error == 0:
            if verbose:
                print(f"Обучение завершено на эпохе {epoch}")
            break

```

```

        return self.loss_history

def print_weights_table(self, title: str = ""):

    if title:
        print(f"\n{title}")

    print("\nТаблица 1. Параметры НС на последовательных эпохах\n")
    print("| Номер эпохи k | Вектор весов w |  

Выходной вектор y | Суммарная ошибка E  

| \n")

    for epoch in range(len(self.loss_history)):
        w = self.weights_history[epoch]
        outputs = self.output_history[epoch]
        error = self.loss_history[epoch]

        w_str = f"({w[0]:.2f}, {w[1]:.2f}, {w[2]:.2f}, {w[3]:.2f},  

{w[4]:.2f})"

        if len(outputs) > 16:
            first_eight = outputs[:16]
            outputs_str = f"({' , '.join(map(str, first_eight))})"
        else:
            outputs_str = f"({' , '.join(map(str, outputs))})"

        print(f"| {epoch:13d} | {w_str:33s} | {outputs_str:29s} |  

{error:18.2f} |")

def print_minimal_table(self, title: str = ""):

    if title:
        print(f"\n{title}")

    print("\nТаблица 3. Параметры НС на последовательных эпохах  

(минимальный набор)\n")
    print("| Номер эпохи k | Вектор весов w |  

Выходной вектор y | Суммарная ошибка E  

| \n")

    for epoch in range(len(self.loss_history)):
        w = self.weights_history[epoch]
        outputs = self.output_history[epoch]
        error = self.loss_history[epoch]

        w_str = f"({w[0]:.2f}, {w[1]:.2f}, {w[2]:.2f}, {w[3]:.2f},  

{w[4]:.2f})"
        outputs_str = f"({' , '.join(map(str, outputs))})"

```

```

        print(f"| {epoch:13d} | {w_str:33s} | {outputs_str:18s} |
{error:18.2f} |")

def boolean_function(x1: int, x2: int, x3: int, x4: int) -> int: # F =
( $\neg x1 + \neg x2 + \neg x3$ )  $\cdot$  ( $\neg x2 + \neg x3 + x4$ )
    # Вычисляем отрицания
    not_x1 = 1 - x1
    not_x2 = 1 - x2
    not_x3 = 1 - x3

    # Первая дизъюнкция:  $\neg x1 + \neg x2 + \neg x3$ 
    term1 = 1 if (not_x1 or not_x2 or not_x3) else 0

    # Вторая дизъюнкция:  $\neg x2 + \neg x3 + x4$ 
    term2 = 1 if (not_x2 or not_x3 or x4) else 0

    # Конъюнкция (логическое умножение)
    return term1 and term2

def generate_truth_table() -> Tuple[List[List[int]], List[int]]: #
Генерация таблицы истинности для всех 16 комбинаций
    X = []
    targets = []

    for i in range(16): # Раскладываем число на биты: x1, x2, x3, x4
        x1 = (i >> 3) & 1
        x2 = (i >> 2) & 1
        x3 = (i >> 1) & 1
        x4 = i & 1

        X.append([x1, x2, x3, x4])
        targets.append(boolean_function(x1, x2, x3, x4))
    return X, targets

def print_truth_table(X: List[List[int]], targets: List[int]):
    print("\nТАБЛИЦА ИСТИННОСТИ\n")
    print(" № | x1 x2 x3 x4 |  $\neg x1$   $\neg x2$   $\neg x3$  | A= $\neg x1 + \neg x2 + \neg x3$  | B= $\neg x2 + \neg x3 + x4$  |
F \n")

    for i in range(16):
        x = X[i]
        not_x1 = 1 - x[0]
        not_x2 = 1 - x[1]
        not_x3 = 1 - x[2]

        term1 = 1 if (not_x1 or not_x2 or not_x3) else 0
        term2 = 1 if (not_x2 or not_x3 or x[3]) else 0
        f = targets[i]

```

```

        print(f"{i:2} |
{x[0]} {x[1]} {x[2]} {x[3]} | {not_x1} {not_x2} {not_x3} | "
              f"      {term1}      |      {term2}      | {f}")

    zeros = sum(1 for t in targets if t == 0)
    ones = sum(1 for t in targets if t == 1)
    print("-"*100)

def run_experiment_full(activation_type: str, X: List[List[int]], targets:
List[int],
                        eta: float, max_epochs: int) -> Neuron:

    act_names = {'threshold': 'Пороговая', 'type2': 'Логическая'}

    print(f"\n1.{1 if activation_type == 'threshold' else 2}. Используя
{act_names[activation_type]} ФА:\n")

    neuron = Neuron(activation_type)
    print(f"Начальные веса: ({neuron.weights[0]:.2f},
{neuron.weights[1]:.2f}, {neuron.weights[2]:.2f}, {neuron.weights[3]:.2f},
{neuron.weights[4]:.2f})")
    print(f"Норма обучения  $\eta$  = {eta}")
    print()
    neuron.train(X, targets, eta, max_epochs, verbose=True,
experiment_type="full")
    print(f"\nИтоговые веса: ({neuron.weights[0]:.2f},
{neuron.weights[1]:.2f}, {neuron.weights[2]:.2f}, {neuron.weights[3]:.2f},
{neuron.weights[4]:.2f})")
    return neuron

def find_minimal_set_threshold(X: List[List[int]], targets: List[int]) ->
Tuple[List[int], Neuron]: # Поиск минимального набора для пороговой
функции

    print(f"\n2.1. Используя пороговую ФА.")
    print("-" * 70)

    # Для варианта 13 минимальный набор из 4 векторов, два представителя
    класса 0 (индексы 14, 15) и два представителя 1
    train_set = [14, 15, 0, 1] # (1,1,1,0), (1,1,1,1), (0,0,0,0),
(0,0,0,1)

    neuron = Neuron('threshold')
    train_X = [X[i] for i in train_set]
    train_t = [targets[i] for i in train_set]

    print(f"Минимальный набор из {len(train_set)} векторов:")
    for i, idx in enumerate(train_set):
        print(f" $X^{\{i+1\}}$  = {train_X[i]}, t = {train_t[i]}")
    print()

```

```

        neuron.train(train_X, train_t, eta=0.3, max_epochs=20, verbose=True,
experiment_type="min")
        print(f"\nДаёт следующие синаптические коэффициенты:")
        print(f"W = ({neuron.weights[0]:.2f}, {neuron.weights[1]:.2f},
{neuron.weights[2]:.2f}, {neuron.weights[3]:.2f},
{neuron.weights[4]:.2f})")
        print(f"Для полного обучения потребовалось {len(neuron.loss_history)}
эпох.")
        return train_set, neuron

def find_minimal_set_type2(X: List[List[int]], targets: List[int]) ->
Tuple[List[int], Neuron]: # Поиск минимального набора для логической
функции

    print(f"\n2.2. Используя логическую функцию\n")
    # Для варианта 13 минимальный набор из 4 векторов, два представителя
класса 0 (индексы 14, 15) и два представителя класса 1
    train_set = [14, 15, 0, 7] # (1,1,1,0), (1,1,1,1), (0,0,0,0),
(0,1,1,1)

    neuron = Neuron('type2')
    train_X = [X[i] for i in train_set]
    train_t = [targets[i] for i in train_set]

    print(f"Минимальный набор из {len(train_set)} векторов:")
    for i, idx in enumerate(train_set):
        print(f"X^{{{i+1}}} = {train_X[i]}, t = {train_t[i]}")
    print()
    neuron.train(train_X, train_t, eta=0.5, max_epochs=30, verbose=True,
experiment_type="min")

    print(f"\nДаёт следующие синаптические коэффициенты:")
    print(f"W = ({neuron.weights[0]:.2f}, {neuron.weights[1]:.2f},
{neuron.weights[2]:.2f}, {neuron.weights[3]:.2f},
{neuron.weights[4]:.2f})")
    print(f"Для полного обучения потребовалось {len(neuron.loss_history)}
эпох.")

    return train_set, neuron

def main():

    X, targets = generate_truth_table() # Генерируем таблицу истинности
    print_truth_table(X, targets)

    print("\n1. ОБУЧЕНИЕ НС С ИСПОЛЬЗОВАНИЕМ ВСЕХ КОМБИНАЦИЙ
ПЕРЕМЕННЫХ\n")

```



```

    neuron1 = run_experiment_full('threshold', X, targets, eta=0.3,
max_epochs=30)
    neuron2 = run_experiment_full('type2', X, targets, eta=0.5,
max_epochs=30)

    print("\n2. ОБУЧЕНИЕ НС С ИСПОЛЬЗОВАНИЕМ ЧАСТИ КОМБИНАЦИЙ
ПЕРЕМЕННЫХ\n")

    min_set1, neuron_min1 = find_minimal_set_threshold(X, targets)
    min_set2, neuron_min2 = find_minimal_set_type2(X, targets)

    print("\nТаблица 1. Параметры НС на последовательных эпохах (пороговая
ФА)\n")
    neuron1.print_weights_table()

    print("\nТаблица 2. Параметры НС на последовательных эпохах (функция
типа 2)\n")
    neuron2.print_weights_table()

    print("\nТаблица 3. Параметры НС на последовательных эпохах (пороговая
ФА, минимальный набор)\n")
    neuron_min1.print_minimal_table()

    print("\nТаблица 4. Параметры НС на последовательных эпохах (функция
типа 2, минимальный набор)\n")
    neuron_min2.print_minimal_table()

if __name__ == "__main__":
    main()

```